Kalenteri - projekti

Milla Granfors 103642556, Tietotekniikka 2025 28.4.2026

2. Yleiskuvaus:

Työssäni loin hyödyllisen graafisen kalenteriohjelman monella erilaisella ominaisuudella. Työ on kirjoitettu Scala-kielellä ja ScalaFX-kirjastolla. Kalenteriin pystyy tallentamaan tapahtumia, sekä ryhmittelemään niitä eri kategorioihin erilaisten aihepiirien mukaisesti (esim. työasiat tai harrastukset). Tapahtuman voi merkitä kestävän monta tuntia tai jopa vuorokautta. Kalenteri voi antaa muistutuksia tapahtumista ja tehtävistä. Se näyttää myös yleiset pyhäpäivät punaisella korostettuna.

Kalenterissa voi mm. poistaa, lisätä ja muokata tapahtumia, käyttää värikoodeja erityyppisten tapahtumien ryhmittelyssä, merkitä kalenterimerkinnälle lisätietoja (kuten, paikka, verkkolinkki, yms.), sekä valita vain tietyn kategorian tapahtumat näkyviin. Värikoodauksessa voi käyttää monipuolisesti kaikkia värejä ja niiden eri sävyjä. Esimerkiksi kaikki opiskeluun liittyvät aiheet voi käyttäjä valita olevan vihreän sävyisiä, luennot tummanvihreällä ja tehtävän palautukset kirkkaan vihreällä.

Kalenterissa on valittavana viikko – ja päivä näkymät, joista käyttäjä saa valita kumpaa hän haluaa käyttää. Näkymän vaihtaminen sujuu helposti oikealta palstalta nappia painamalla. Jokaisessa sarakkeessa näkyy vuorokauden kaikki tunnit. Oletuksena avautuu tämänhetkinen viikko ja käyttäjä voi selata kalenteria viikko kerrallaan oikeassa yläkulmassa sijaitsevien nuolien avulla.

Toteutin työn keskivaikeana tehtävänä. Alkuperäiseen suunnitelmaan nähden ohjelmassa on monipuoliset muokkausominaisuudet. Käyttäjä pystyy muokkaamaan jo luotuja tapahtumia ja kategorioita. Lisäksi kalenteriin pystyy lisäämään automaattisia muistutuksia. Puuttumaan jäi valitettavasti ajanpuutteen takia hiirellä maalausominaisuus, sekä tapahtumien hakeminen.

3. Käyttöohje:

Ohjelma käynnistetään ajamalla projektin pääluokka (Main.scala). Ohjelma lataa käynnistäessään aiemmin tallennetut tapahtumat ja kategoriat automaattisesti sille annetusta tiedostosta. Ohjelman käynnistyessä näytölle aukeaa kalenteri, jota on helppo käyttää.

Toiminnot:

- **Navigointi:** Oikeassa yläkulmassa olevista nuoli näppäimistä (> ja < - painikkeet) voi siirtyä viikon tai päivän kerrallaan eteen- tai taaksepäin niitä painamalla.

- **Tapahtuman lisääminen:** Navigointi – painikkeiden alapuolella oikealla palstalla on + - painike, jota painamalla avautuu tapahtuman luomista varten dialogi. Dialogiin pystyy syöttämään monia tietoja tapahtumasta, mutta pakollisia tietoja syöttää ovat vain nimi, sekä aloitus ja lopetus – päivät ja kellonajat. Päivämäärät tulee syöttää muodossa: PP.KK.VVVV ja kellonajat: HH:mm. Lisäksi voi mm. valita tapahtuman kategorian (se on oletuksena ”Other”), lisätä tapahtumapaikan, muut osallistujat, verkkolinkin tai muistutuksen. Dialogissa on näkyvillä ohjeet, miten tiedot tulee syöttää.
- **Tapahtuman muokkaaminen ja poistaminen:** Jo valmiiksi lisättyjä tapahtumia pystyy muokkaamaan ja poistamaan klikkaamalla niitä kalenterinäkymässä. Tällöin aukeaa sama dialogi, jonka avulla tiedot tallennettiin tapahtumaan. Niitä pystyy muokkaamaan, jonka jälkeen ne voi tallentaa ”Save” -painikkeella tai poistaa kokonaan ”Delete”- painikkeella.
- **Näkymän suodatus**: Oikeassa reunassa on lista kategorioita (esim. Work, School, Other). Klikkaamalla valintaruutuja voi piilottaa tai näyttää tiettyyn kategoriaan kuuluvat tapahtumat välittömästi. Kategorioita voi olla valittuna useampi tai vain yksi samaan aikaan.
- **Kategorioiden hallinta:** ”New Category” – painikkeella voi luoda uuden kategorian ja valita sille nimen ja värin väripaletista aukeavan dialogin avulla. Lisäksi jokaisen kategorian vieressä on ”Edit” – painike, josta painamalla sama dialogi jälleen avautuu ja kategorian tietoja pystyy mielivaltaisesti muuttamaan tai poistamaan kokonaan.
- **Näkymän vaihto:** Näkymää pystyy vaihtamaan ”Week” – ja ”Day” – painikkeista. Nimensä mukaisesti ”Week”- painike avaa viikkonäkymän ja ”Day”-painike päivänäkymän.
- **Muistutukset**: Ohjelma tarkistaa automaattisesti taustalla, onko lähiaikana alkamassa tapahtumia, joihin on laitettu muistutus. Muistutuksen tapahtumalle voi asettaa tapahtuman dialogi- ruudulla, jossa kysytään minuutteina, kuinka kauan ennen tapahtuman alkua käyttäjä haluaa muistutuksen. Muistutukset tulevat näkyviin ”Notifications” – listalle.

4. Ohjelman rakenne:

Tein alla olevan UML - kaavion. Siinä on eroteltu ohjelman tärkeimmät osakokonaisuudet ja luokkajaot. Alkuperäisestä suunnitelmastani luokkien määrä väheni rajusti, jonka huomaa hyvin UML – kaaviosta. Ohjelma noudattaa nyt paljon selkeämpää jakoa käyttöliittymän ja sovelluslogiikan välillä.

Ohjelman yksi tärkein luokka on CalendarView–luokka, joka on ohjelman pääasiallinen graafinen käyttöliittymä (GUI), joka perii BorderPane nimisen luokan. Se kokoaa kalenterin ulkoasun ja on se mitä käyttäjä näkee. Esimerkiksi CalendarView -luokan

kautta muodostuu tapahtumien lisäämiseen tarvittava dialogi näkymä. CalendarView – luokka käyttää Calendar – luokkaa datan hakemiseen.

Calendar – luokka on puolestaan ohjelman keskusyksikkö, joka hallitsee tapahtumien kokoelmaa. Se kokoaa kalenterin ja tallentaa näin tiedot avulla tiedostoon. Se tarjoaa metodit mm. tapahtumien hakemiseen ja muistutusten tarkistamiseen. Calendar–luokassa on listattuna kaikki eri tapahtumat Event - olioina, jotka ovat lisätty kalenteriin.

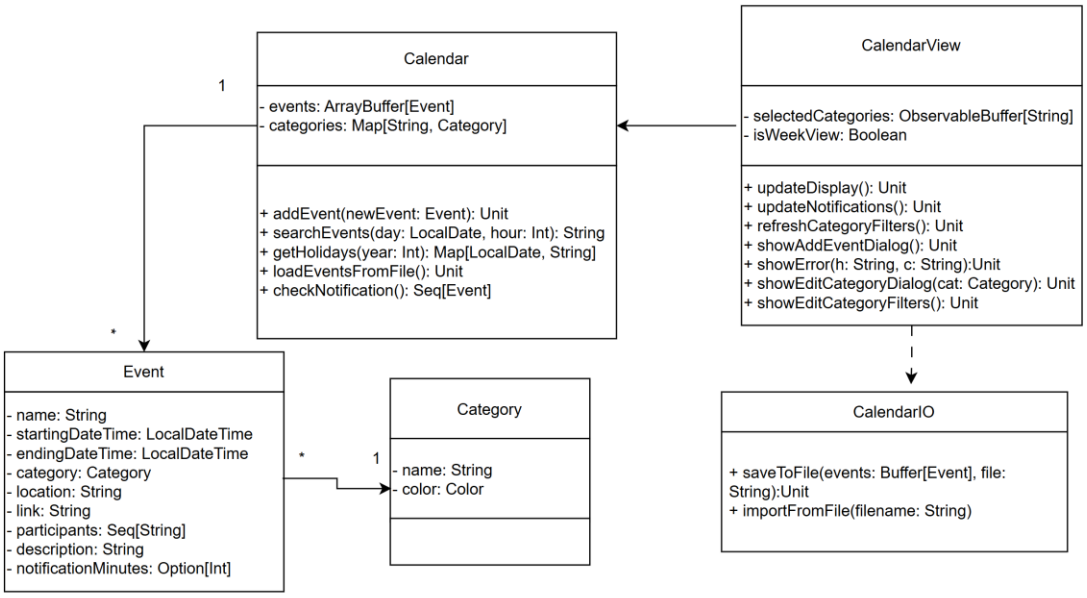
Event – luokka on kaikista tarkin, joka mallintaa yksittäistä kalenteritapahtumaa. Siihen merkitään kaikkiin tapahtumiin liittyvät tiedot, kuten nimi, aloitusaika, lopetusaika, sekä mahdolliset lisätiedot (paikka, henkilöt, nettilinkit, yms.) Event–oliolla on assosiaatio yhteen Category–olioon.

Category- luokka puolestaan ryhmittelee tapahtumia käyttäjän valitsemalla tavalla. Ryhmittely tapahtuu jonkun aihepiirin ympärille, kuten esimerkiksi. opiskelu tai työ. Jokaisella ryhmällä on oma nimensä ja värinsä. Kategoriat mahdollistavat kalenterin visuaalisen jäsentämisen ja suodattamisen.

CalendarIO – objekti vastaa tiedon pysyvyydestä. Se lukee ja kirjoittaa tapahtumatiedot tiedostoon, varmistaen että tiedot säilyvät ohjelman sulkemisen jälkeen.

Valitsin tämän rakenteen, koska se oli mielestäni yksinkertaisin ja itselleni aloittelijan näkökulmasta helpointa toteuttaa.

UML – kaavio:



5. Algoritmit:

Ohjelman toiminta perustuu useisiin aikadataa ja kokoelmia käsitteleviin algoritmeihin. Tässä koottuna muutamia tilanteita, joissa hyödynnetään algoritmeja:

Tapahtumien suodatus ja haku (Lineaarinen haku):

Kun kalenterinäkymä piirretään (updateDisplay – metodi), ohjelma käy läpi kaikki viikon tunnit ja päivät. Jokaisen kohdalla suoritetaan haku. Ja metodi vastaa siitä, että oikeat tapahtumat näkyvät oikeissa kohdissa.

Algoritmi: Käydään läpi kaikki tapahtumat ja tarkistetaan, osuuko tapahtuman aika valitulle tunnille ja onko tapahtuman kategoria valittuna suodatinlistalla, eli voiko tapahtuma näkyä kalenterinäkymällä tällä hetkellä.

Perustelu: Tämä lineaarinen haku on varma tapa varmistaa, että näkymä on aina ajan tasalla, kun käyttäjä vaihtaa viikkoa tai muuttaa kategorian suodatinasetuksia.

Reaaliaikainen muistutusten seuranta:

Ohjelma seuraa taustalla jatkuvasti ajan kulumista ilmoituksia varten.

Algoritmi: Joka sekunti haetaan nykyinen kellonaika, käydään läpi kaikki kalenterin tapahtumat ja jos tapahtuman alkuaika vähennettynä nykyhetkestä on pienempi tai yhtä suuri kuin tapahtumalle asetettu muistutusaika lisätään tapahtuma näkyviin ilmoituslistalle.

Perustelu: Käyttämällä Timeline – ajastinta vältetään ohjelman ”jäätyminen” ja käyttäjä saa tiedon lähestyvistä menoista ilman sivun päivitystä.

6. Tietorakenteet:

Ohjelmassa käytetään pääasiassa Scalan valmiita kokoelmatyyppejä, jotka on valittu niiden tarjoaman joustavuuden ja tehokkuuden perusteella.

Tapahtumien tallennukseen käytän Scalan ArrayBuffer[Event]–rakennetta. Koen, että se sopii projektiini parhaiten. Se sallii dynaamisen koon muuttamisen.

Tapahtumien aiheryhmien tallennukseen käytän Map[String, Category]–rakennetta, jossa avain = ryhmän nimi ja arvo = Category-olio. Tämä mahdollistaa mm. nopean haun.

7. Tiedostot ja verkossa oleva tieto:

Ohjelma tallentaa tiedot tekstitiedostoon (kalenteri.ics), joka noudattaa yksinkertaistettua versiota standardista iCalendar-formaatista. Se on suosittu kalenterien keskuudessa, jonka takia päädyin valitsemaan sen. iCalendar mahdollistaa tietojen lukemisen millä tahansa tekstieditorilla.

Tässä esimerkki tiedostosta, jonka ohjelma tallentaa:

```
BEGIN:VCALENDAR
PRODID:-//Milla//FI
VERSION:2.0;2.0
BEGIN:VEVENT
DTSTAMP:20260428T201948Z
DTSTART:20260429T050000Z
DTEND:20260429T070000Z
SUMMARY:Matikan luento
CATEGORIES:School
LOCATION:kandikeskus
ATTENDEE:Isabella
ATTENDEE:Erika
DESCRIPTION:muista laskin!
X-REMINDER-MINS:30
END:VEVENT
END:VCALENDAR

```

Ohjelma lukee tiedoston rivi kerrallaan ja muodostaa niistä Event-olioita.

8. Testaus:

Erillisiä yksikkötestiluokkia ei rakennettu, vaan testaus eteni "koodaa ja kokeile"-periaatteella kunkin uuden ominaisuuden jälkeen. Tämä poikkesi suunnitelmasta, jossa kaavailin automaattisia testejä, mutta graafisen käyttöliittymän monimutkaisuus teki manuaalisesta testauksesta tehokkaampaa tässä vaiheessa.

GUI:ta testasin todella monta kertaa sitä luodessa. Jokaista ominaisuutta luodessa oli erityisen tärkeää etenkin tämän tyyppisessä projektissa testata, miltä se tulee oikeasti näyttämään. Testasin esimerkiksi, että tapahtumat piirtyvät oikeille kohdille, suodattimet piilottavat oikeat kategoriat ja että dialogit validioivat syötteet (esim. tyhjä nimi tai virheellinen kellonaika).

Tallentamista testasin luomalla uusia tapahtumia kalenterin ja tarkistamalla, että ne tallentuvat. Testasin myös esimerkiksi, että kalenterissani toimii usean päivän yli kestävän tapahtuman luominen ja varmistin myös, ettei ohjelma kaadu päällekkäisiin tapahtumiin.

9. Ohjelman tunnetut puutteet ja viat:

Kaikkeen ei valitettavasti riittänyt aika, joten puutteitakin jäi ohjelmaan. Paremmalla ajalla niitäkin voisi vielä parannella:

- **Päällekkäiset tapahtumat:** Ohjelmassa päällekkäiset tapahtumat piirtyvät toistensa päälle. Parannus olisi laskea tällaisessa tilanteessa tapahtumien leveys uudelleen niin, että ne näkyisivät kalenterissa vierekkäin.
- **Muistutusten asettaminen muuten kuin minuutteja käyttäen:** Jos käyttäjä haluaa asettaa muistutuksen muutaman päivän päähän, hänen täytyy tällä hetkellä laskea se minuutteina. Parannuksena tekisin esimerkiksi valintaruudut, joista käyttäjä saisi valita millä yksiköllä ajan ilmoittaa.
- **Tapahtumien hakeminen:** Tämä ominaisuus jäi minulta ajanpuutteen takia kokonaan tekemättä.
- **Hiirellä maalaaminen:** Ominaisuus, jossa tapahtumia voisi lisätä kalenteriin pelkästään hiirellä maalaamalla jäi minulta nyt tekemättä.

10. 3 parasta ja 3 heikointa kohtaa:

3 parasta:

- Dynaaminen suodatus: Kategorioiden piilottaminen CheckBoxeilla toimii välittömästi ja parantaa käyttökokemusta huomattavasti. Lisäksi ne näyttävät ulkonäöllisesti miellyttäviltä.
- Kategorioiden muokattavuus: Mahdollisuus vaihtaa jo luotujen kategorioiden värejä ja nimiä tai mahdollisesti niiden poistaminen tekee ohjelmasta miellyttävämmän käyttää.
- Visuaalinen selkeys: Panostin ehdottomasti tässä projektissa kalenteri näytön ulkonäköön. Käytin erilaisia värejä, fontteja ja kokoja, joka tekevät mielestäni kalenterista modernin ja selkeän näköisen.

Bonus: Juhlapyhien luominen, ja erityisesti Pääsiäispyhien laskeneminen.

3 heikointa:

- Koodin pituus: CalendarView – luokka on kasvanut melko suureksi ja muutama metodi tekee paljon työtä, jotka olisi voinut myös jakaa muutaman metodin kesken. Se olisi tuonut selkeyttä koodiin.
- Tapahtumien lisäämisen työläys: Tapahtuminen lisääminen vaatii käyttäjän kirjoittavan tapahtuman ajankohdat päivämäärineen ja kellonaikoineen manuaalisesti. Siinä on yleensä vähän toistoa, jos tapahtuma on yhden päivän aikana.
- Päälekkäisten tapahtumien asettelu: Jos kaksi tai useampi tapahtuma on samaan aikaan, ne piirtyvät päällekkäin, jolloin toinen peittyy.

11. Poikkeamat suunnitelmasta, toteutunut työjärjestys ja aikataulu:

Projektin toteutus noudatti pääpiirteissään alkuperäistä suunnitelmaa, mutta työn edetessä piti projektin kanssa ottaa loppukiri. Työ toteutettiin vaiheissa, joissa uudet ominaisuudet testattiin välittömästi niiden valmistuttua.

Toteutunut työjärjestys suurin piirtein:

- Ensimmäinen vaihe (Perusrakenne): Työ aloitettiin luomalla keskeisimmät tietorakenteet (Event, Calendar) ja perusnäkymä. Tavoitteena oli saada kalenteriruudukko näkyviin ja perusnavigointi toimimaan.
- Toinen vaihe (Toiminnallisuus): Seuraavaksi keskityttiin tapahtumien lisäämiseen ja muokkaamiseen dialogien kautta. Tässä vaiheessa toteutettiin myös tiedostoon tallentaminen (CalendarIO).
- Kolmas vaihe (Hienosäätö ja lisäominaisuudet): Viimeisessä vaiheessa keskityttiin visuaaliseen ilmeeseen, kategorioiden dynaamiseen suodatukseen ja juhlapyhäalgoritmin kaltaisiin yksityiskohtiin.

Opin työtä tehdessä valtavan paljon. Prosessin aikana opin eniten modulaarisen koodin merkityksestä. Huomasin, että mitä pidemmälle projekti eteni, sitä tärkeämpää oli pitää käyttöliittymä ja logiikka erillään. Lisäksi opin paljon Scalan eri kokoelmista ja, miten niitä käytetään. Ylipäätänsä tämä oli ihan ensimmäinen kerta, kun loin alusta loppuun näin paljon itse. Jos tekisin projektin uudelleen panostaisin ajankäyttöön, jotta aikaa olisi huolellisesti miettiä, mitkä oikeasti olisivat parhaimpia koodaus ratkaisuja.

12. Kokonaisarvio lopputuloksesta:

Ohjelma saavutti sille asetetut tavoitteet ja täyttää suurin piirtein keskivaikean tason vaatimukset. Mielestäni suurin vahvuus on visuaalinen selkeys ja käytettävyys. Projektissa on muutamia vikoja, mutta kaikki tarvittavat toiminnot toimivat oikein hyvin. Vaikka koodin modulaarisuudessa ja päällekkäisten tapahtumien asettelussa on parannettavaa, iCalendar-yhteensopivuus tekee siitä toimivan kalenterin. Projekti opetti valtavasti ScalaFX-kirjaston hallinnasta ja opetti modulaarisen koodin tärkeyden käytännössä. Jos aloittaisin alusta, jakaisin GUI-komponentit pienempiin luokkiin heti alussa. Olen tyytyväinen tähän, mitä sain aikaiseksi.

13. Viitteet ja muualta otettu koodi:

14. Liitteet:

Algoritmeista tietoa: https://www.ohjelmointiputka.net/oppaat/opas.php?tunnus=alg_2

Scala dokumentaatio:

- https://docs.scala-lang.org/
- https://docs.scala-lang.org/getting-started/intellij-track/testing-scala-in-intellij-with-scalatest.html (scalatest)

Muita linkkejä, joista saanut tietoa.

https://scalafx.org/docs/dialogs_and_alerts/

https://gist.github.com/sentenza/e8f8003e2581d52f12f8c89d663c00d7

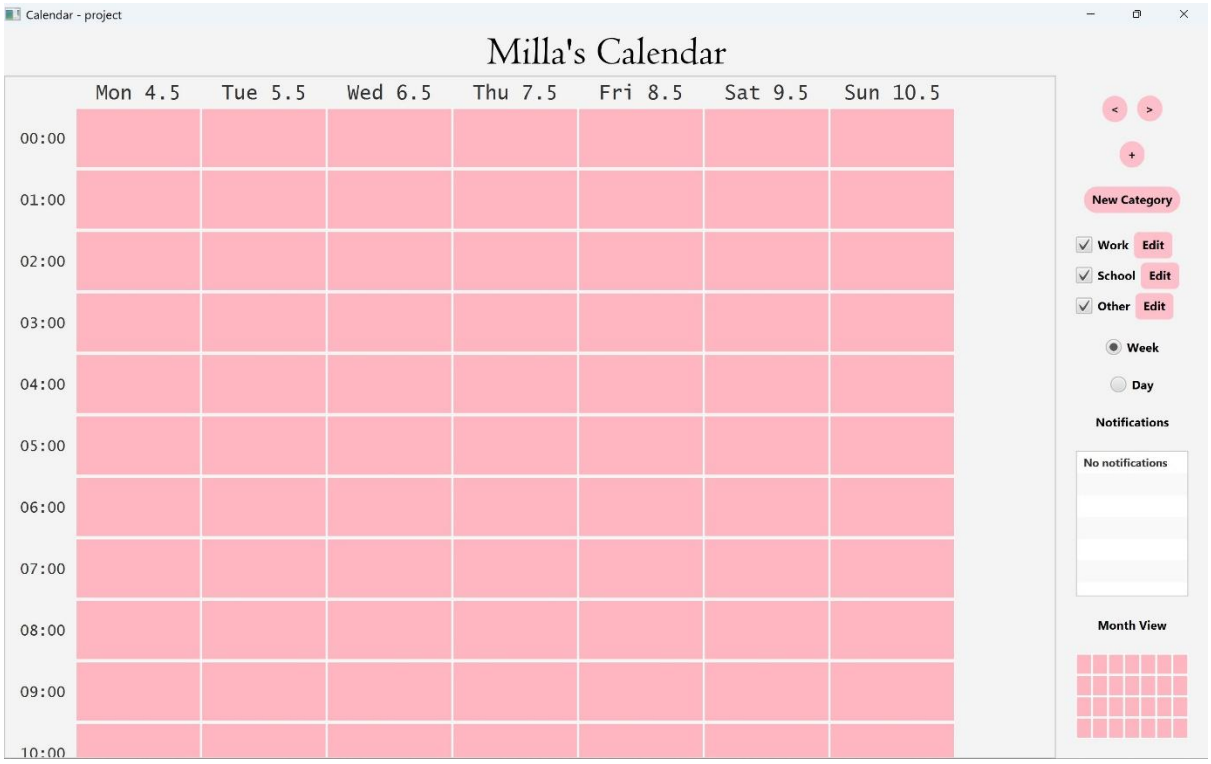
https://stackoverflow.com/questions/71917654/how-can-i-complete-this-python-program-that-implements-the-anonymous-gregorian-c

https://stackoverflow.com/questions/28630476/scalafx-animation-timeline-not-working-as-expected

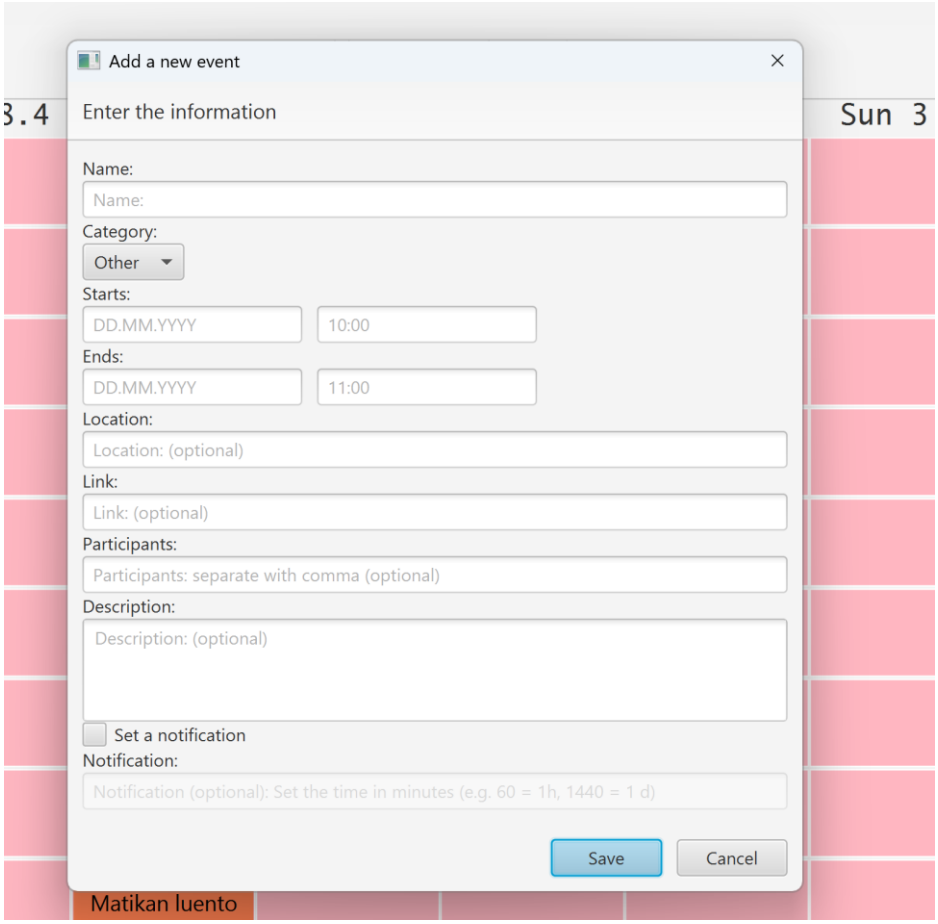
Gemini AI, apuna koodin ymmärtämisessä ja opettamisessa

Kuvia projektin toiminnasta:

Kalenterin näyttö:



Tapahtuman lisäys dialogi:



Kuvassa näkyvissä, että vain yhden kategorian tapahtumat näkyvissä:

